Lab Handout 2-B

Universal Asynchronous Receiver/Transmitter (UART) Receiver with Oversampling and Clock-Domain Synchronizer

Capstone — Module 2: Field-Programmable Gate Array (FPGA) Register-Transfer Level (RTL) Design and Signal Ingestion

1 Objective

Replace the bring-up `uart_rx` from LH1-H with a generic-parameterized 115200-baud UART receiver and add a unit testbench plus a regression test against recorded Module 1 capture data. By the end of this handout your project will contain a fully verified `uart_rx.vhd` that emits a one-cycle `rx_strobe` pulse per received byte, plus two testbenches in the `sim_1` fileset.

2 Prerequisites

- Lab Handout 2-A (LH1-H project open, simulation fileset created, recorded Module 1 `.bin` capture in the `data/` folder).

3 Background

The STM32 interceptor drives the UART line asynchronously; its transmit clock has no relationship to the FPGA's 100 MHz `sysclk`. The UART receiver IP block therefore performs two jobs: (1) cross `uart_rxd` safely into the `sysclk` domain, and (2) recover byte boundaries by oversampling.

Why 16× oversampling?

Sampling each bit 16 times per bit-period and taking the middle sample tolerates about ±3% of baud-rate mismatch between the transmitter and the FPGA UART receiver. At 115200 baud with a 100 MHz system clock, the oversample tick period is $100\,\text{MHz}/(115200 \times 16) \approx 54$ cycles. A simple counter dividing `sysclk` by 54 produces the sample-enable strobe used by the receiver Finite State Machine (FSM).

What changes versus the LH1-H version?

The LH1-H bring-up `uart_rx` hard-codes 100 MHz and 115200 baud as integer literals and folds the two-flop synchronizer into the same process as the FSM. The Module 2 version below: (1) parameterizes `CLK_FREQ_HZ` and `BAUD` as generics so the same file is reusable on a different board or at a different baud, and (2) widens the oversample counter to 16 bits so the generic divider value at higher `CLK_FREQ_HZ` fits without modification. The entity port list (`clk`, `rst_n`, `rx`, `rx_data`, `rx_strobe`) is identical to LH1-H so `ingest_top.vhd` compiles without changes.

4 Procedure

4.1 Step 1 — Save a reference copy of the LH1-H `uart_rx`

The Module 2 `uart_rx` keeps the LH1-H entity name and port list, so it is edited in place rather than removed. Before editing, copy the existing file aside so you have a reference for the LH1-H-vs-Module-2 comparison required in your deliverable:

1. Outside Vivado, copy `ingest_top.srcs/sources_1/new/uart_rx.vhd` to a backup location of your choice (for example `ingest_top/reference/uart_rx_lh1h.vhd`). Do not add the backup to the Vivado project.

4.2 Step 2 — Edit `uart_rx.vhd` in place

1. In Vivado's Sources panel, double-click the existing `uart_rx` entry under Design Sources.
2. Select all (`Ctrl+A`), delete the LH1-H contents, and paste the listing below verbatim. Save (`Ctrl+S`).

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity uart_rx is
    generic (
        CLK_FREQ_HZ : integer := 100_000_000;
        BAUD        : integer := 115_200
    );
    port (
        clk        : in  std_logic;
        rst_n      : in  std_logic;
        rx         : in  std_logic;
        rx_data    : out std_logic_vector(7 downto 0);
        rx_strobe  : out std_logic
    );
end uart_rx;

architecture rtl of uart_rx is
    constant DIV16 : integer := CLK_FREQ_HZ / (BAUD * 16);  -- 54 at defaults
    signal os_cnt  : unsigned(15 downto 0) := (others => '0');
    signal os_tick : std_logic := '0';

    type state_t is (S_IDLE, S_START, S_DATA, S_STOP);
    signal state    : state_t := S_IDLE;
    signal bit_idx  : unsigned(2 downto 0) := (others => '0');
    signal sub_cnt  : unsigned(3 downto 0) := (others => '0');
    signal shifter  : std_logic_vector(7 downto 0) := (others => '0');
    signal rx_sync1, rx_sync2 : std_logic := '1';
begin
    process (clk) begin
        if rising_edge(clk) then
            rx_sync1 <= rx;
```

```
            rx_sync2 <= rx_sync1;
        end if;
    end process;

    process (clk) begin
        if rising_edge(clk) then
            if rst_n = '0' then
                os_cnt  <= (others => '0');
                os_tick <= '0';
            elsif os_cnt = to_unsigned(DIV16 - 1, os_cnt'length) then
                os_cnt  <= (others => '0');
                os_tick <= '1';
            else
                os_cnt  <= os_cnt + 1;
                os_tick <= '0';
            end if;
        end if;
    end process;

    process (clk) begin
        if rising_edge(clk) then
            rx_strobe <= '0';
            if rst_n = '0' then
                state   <= S_IDLE;
                sub_cnt <= (others => '0');
                bit_idx <= (others => '0');
            elsif os_tick = '1' then
                case state is
                    when S_IDLE =>
                        if rx_sync2 = '0' then
                            state   <= S_START;
                            sub_cnt <= to_unsigned(0, 4);
                        end if;
                    when S_START =>
                        if sub_cnt = to_unsigned(7, 4) then
                            if rx_sync2 = '0' then
                                state   <= S_DATA;
                                sub_cnt <= (others => '0');
                                bit_idx <= (others => '0');
                            else
                                state <= S_IDLE;
                            end if;
                        else
                            sub_cnt <= sub_cnt + 1;
                        end if;
                    when S_DATA =>
                        if sub_cnt = to_unsigned(15, 4) then
                            shifter <= rx_sync2 & shifter(7 downto 1);
                            sub_cnt <= (others => '0');
                            if bit_idx = to_unsigned(7, 3) then
                                state <= S_STOP;
                            else
                                bit_idx <= bit_idx + 1;
                            end if;
```

```
                        else
                            sub_cnt <= sub_cnt + 1;
                        end if;
                    when S_STOP =>
                        if sub_cnt = to_unsigned(15, 4) then
                            rx_data   <= shifter;
                            rx_strobe <= '1';
                            state     <= S_IDLE;
                        else
                            sub_cnt <= sub_cnt + 1;
                        end if;
                end case;
            end if;
        end if;
    end process;
end rtl;
```

Listing 1: `uart_rx.vhd` (Module 2) — 115200 8N1 receiver, 16× oversampled, with `CLK_FREQ_HZ` and `BAUD` generics. Port list matches LH1-H exactly.

4.3 Step 3 — Confirm `ingest_top` still elaborates

1. Right-click `ingest_top` in Sources → **Hierarchy Update** → **Automatic Update, Manual Compile Order**. The red "!" marker should clear.

2. **Confirm `ingest_top` is set as the Design Sources top.** In the Sources panel's *Design Sources* tab, `ingest_top` should appear in **bold**. If not, right-click `ingest_top` → **Set as Top**. Synthesis uses the Design Sources top regardless of what is set as top in Simulation Sources; verify it explicitly before every synthesis run.

3. Re-run synthesis. Expect zero errors. (You will see a new Vivado info note that `uart_rx` now has generics with their default values bound.)

4. Optionally re-program the board and confirm the Beacon ID display still updates — your Module 2 implementation is now live in the end-to-end path.

4.4 Step 4 — Write the unit testbench

1. Sources panel: + → **Add or create simulation sources** → **Create File**. File type **VHDL**, name `tb_uart_rx`. Click **OK**, **Finish**, and dismiss the Define Module dialog.

2. Double-click the new `tb_uart_rx` entry. Replace the entire contents with the listing below and save.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use STD.TEXTIO.ALL;

entity tb_uart_rx is
end tb_uart_rx;

```

```
architecture sim of tb_uart_rx is
    constant CLK_PERIOD : time    := 10 ns;          -- 100 MHz
    constant BIT_PERIOD : time    := 8680 ns;        -- 1/115200

    signal clk       : std_logic := '0';
    signal rst_n     : std_logic := '0';
    signal rx        : std_logic := '1';             -- idle high
    signal rx_data   : std_logic_vector(7 downto 0);
    signal rx_strobe : std_logic;

    signal strobe_count : integer := 0;
    signal last_byte    : std_logic_vector(7 downto 0) := (others => '0');

    -- Local hex formatter. Vivado xsim does not always resolve to_hstring
       ()
    -- for std_logic_vector under the default VHDL version, so build the
       hex
    -- string by hand to keep the testbench portable.
    function slv_to_hex (v : std_logic_vector(7 downto 0)) return string
       is
        constant HEX : string(1 to 16) := "0123456789ABCDEF";
        variable hi  : integer := to_integer(unsigned(v(7 downto 4)));
        variable lo  : integer := to_integer(unsigned(v(3 downto 0)));
    begin
        return HEX(hi + 1) & HEX(lo + 1);
    end function;

    procedure send_byte (signal s : out std_logic; b : in std_logic_vector
       (7 downto 0)) is
    begin
        s <= '0'; wait for BIT_PERIOD;                -- start
        for i in 0 to 7 loop
            s <= b(i); wait for BIT_PERIOD;           -- LSB first
        end loop;
        s <= '1'; wait for BIT_PERIOD;                -- stop
    end procedure;
begin
    -- Clock generator
    clk <= not clk after CLK_PERIOD / 2;

    -- DUT
    dut : entity work.uart_rx
        port map (clk => clk, rst_n => rst_n,
                  rx => rx, rx_data => rx_data, rx_strobe => rx_strobe);

    -- Strobe counter / capture
    process (clk) begin
        if rising_edge(clk) then
            if rx_strobe = '1' then
                strobe_count <= strobe_count + 1;
                last_byte    <= rx_data;
            end if;
        end if;
    end process;
```

```vhdl

    -- Stimulus
    process
    begin
        wait for 100 ns;
        rst_n <= '1';
        wait for 1 us;

        send_byte(rx, x"A5");
        wait for 200 us;                                -- generous settle

        assert strobe_count = 1
            report "expected exactly one rx_strobe pulse, got "
                   & integer'image(strobe_count) severity failure;
        assert last_byte = x"A5"
            report "expected rx_data=0xA5, got 0x"
                   & slv_to_hex(last_byte) severity failure;

        report "tb_uart_rx PASS" severity note;
        std.env.finish;
    end process;
end sim;
```

Listing 2: `tb_uart_rx.vhd` — bit-bangs the byte `0xA5` at 115200 baud and asserts that `rx_strobe` pulses exactly once with `rx_data = 0xA5`.

3. In the Sources panel, right-click `tb_uart_rx` (under Simulation Sources) → **Set as Top**. The simulation fileset's top should now be `tb_uart_rx`.

4. Flow Navigator → **Run Simulation** → **Run Behavioral Simulation**. Vivado launches with the default `1us` runtime, which stops short of the assertions. In the Tcl console at the bottom, type `run 1 ms` and press Enter (*exactly once* — a second `run` call re-enters the stimulus and trips the strobe-count assertion). The Tcl console should then print `tb_uart_rx PASS` at approximately 288 us simulated.

4.5 Step 5 — Regression against recorded Module 1 data

This testbench reads a binary capture from `data/` and bit-bangs every byte through the receiver, comparing each captured byte to the source file. Add the file the same way as Step 4 (+ → **Add or create simulation sources** → **Create File**, name `tb_uart_rx_regression`).

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use STD.TEXTIO.ALL;

entity tb_uart_rx_regression is
end tb_uart_rx_regression;

architecture sim of tb_uart_rx_regression is
    constant CLK_PERIOD   : time   := 10 ns;
    constant BIT_PERIOD   : time   := 8680 ns;
```

[Figure placeholder]

LH2B_waveform.png

Expected waveform: `rx` start-bit fall, eight data bits, stop bit, and a single `rx_strobe` pulse aligned with `rx_data = 0xA5`.

Figure 1: Expected waveform: `rx` start-bit fall, eight data bits, stop bit, and a single `rx_strobe` pulse aligned with `rx_data = 0xA5`.

```
    constant CAPTURE_FILE : string := "../../../../data/capture_clean.bin"; -- CAPTURE_FILE

    signal clk       : std_logic := '0';
    signal rst_n     : std_logic := '0';
    signal rx        : std_logic := '1';
    signal rx_data   : std_logic_vector(7 downto 0);
    signal rx_strobe : std_logic;

    signal expected_byte : std_logic_vector(7 downto 0) := (others => '0');
    signal byte_count    : integer := 0;
    signal mismatch_cnt  : integer := 0;

    type charfile is file of character;

    procedure send_byte (signal s : out std_logic; b : in std_logic_vector(7 downto 0)) is
    begin
        s <= '0'; wait for BIT_PERIOD;
        for i in 0 to 7 loop
            s <= b(i); wait for BIT_PERIOD;
        end loop;
        s <= '1'; wait for BIT_PERIOD;
    end procedure;
begin
    clk <= not clk after CLK_PERIOD / 2;

    dut : entity work.uart_rx
        port map (clk => clk, rst_n => rst_n,
                  rx => rx, rx_data => rx_data, rx_strobe => rx_strobe);

    -- Compare each strobed byte against the most recently transmitted byte.
    process (clk) begin
```

```vhdl
        if rising_edge(clk) then
            if rx_strobe = '1' then
                if rx_data /= expected_byte then
                    mismatch_cnt <= mismatch_cnt + 1;
                end if;
                byte_count <= byte_count + 1;
            end if;
        end if;
    end process;

    process
        file     fh   : charfile;
        variable ch   : character;
        variable byte : std_logic_vector(7 downto 0);
        variable open_status : file_open_status;
    begin
        wait for 100 ns;
        rst_n <= '1';
        wait for 1 us;

        file_open(open_status, fh, CAPTURE_FILE, read_mode);
        assert open_status = open_ok
            report "Could not open capture file " & CAPTURE_FILE severity failure;

        while not endfile(fh) loop
            read(fh, ch);
            byte := std_logic_vector(to_unsigned(character'pos(ch), 8));
            expected_byte <= byte;
            send_byte(rx, byte);
        end loop;

        file_close(fh);
        wait for 200 us;

        assert mismatch_cnt = 0
            report "regression: " & integer'image(mismatch_cnt)
                   & " byte mismatches over " & integer'image(byte_count) severity failure;
        report "tb_uart_rx_regression PASS: " & integer'image(byte_count) & " bytes" severity note;
        std.env.finish;
    end process;
end sim;
```

Listing 3: `tb_uart_rx_regression.vhd` — streams every byte from a recorded `.bin` and asserts byte-for-byte recovery. Adjust the relative path on line marked CAPTURE_FILE if your data folder is elsewhere.

1. Set `tb_uart_rx_regression` as the simulation top.
2. **Increase the simulation runtime before launching.** Each byte takes 87 μs of simulated time; a 63 kB capture needs ~5.5 simulated seconds. In Project Settings → Simulation, set

`xsim.simulate.runtime` to `6sec`. (Without this, Vivado launches with the 1 us default and the stimulus process never reaches the end of the file.)

3. Flow Navigator → **Run Simulation** → **Run Behavioral Simulation**. Wall-clock runtime is roughly 4–6 minutes for a 1500-frame capture; `xsim` bit-bangs at about 18 ms simulated per wall-clock second on a typical laptop. If you'd rather not bump the default, leave it at `1us` and type `run 6 sec` once in the Tcl console after the launch completes.

4. Confirm the Tcl console prints `tb_uart_rx_regression PASS` with the byte count matching the file size on disk (e.g. 62952 bytes for a 1500-frame capture).

5 Verification Checklist

- ☐ `uart_rx.vhd` edited in place to the Module 2 generic-parameterized version; LH1-H copy saved aside as a reference.
- ☐ Two-flop synchronizer present and synthesizes to exactly two flip-flops on `rx`.
- ☐ Oversample tick period is 54 cycles of `sysclk` at the default generics.
- ☐ `tb_uart_rx` prints `PASS`.
- ☐ `tb_uart_rx_regression` prints `PASS` with zero byte mismatches over the entire capture file.
- ☐ Re-synthesis of `ingest_top` produces zero errors and the seven-segment display still updates on hardware.

Common Pitfalls

- **Sampling at the bit edge:** the FSM must sample at mid-bit (tick 8 of 16), not at tick 0 or 15. The listing above samples at `sub_cnt = 7` (start-bit verification) and `sub_cnt = 15` (post-bit settle); do not change these constants.
- **Wrong file path in the regression testbench:** Vivado runs simulations from `ingest_top.sim/sim_1/behav/xsim/` by default, so the path to `data/` from there is `../../../../data/capture_clean.bin` (four `..` to climb out of the `xsim/`, `behav/`, `sim_1/`, and `ingest_top.sim/` folders to reach the project root). Adjust the `CAPTURE_FILE` constant if your project layout differs, or use an absolute path during initial bring-up. The regression runs against `capture_clean.bin`; `capture_mixed.bin` is reserved for LH2-G's end-to-end testbench, where its button-hold gaps stress the parser's re-sync logic.
- **Wrong bit order:** the LSB arrives first on the wire, not the MSB. The shifter line `shifter <= rx_sync2 & shifter(7 downto 1)` is correct; do not invert it.

Lab Handout 2-B Deliverable

1. Module 2 `uart_rx.vhd`, `tb_uart_rx.vhd`, and `tb_uart_rx_regression.vhd`.
2. Tcl console transcript or screenshot showing both testbenches printing `PASS`.
3. Waveform screenshot of the `0xA5` unit test.
4. One-paragraph note comparing the LH1-H and Module 2 `uart_rx` files (kept aside per Step 1)

— specifically, what changed and why.

Glossary

FPGA	Field-Programmable Gate Array	1
FSM	Finite State Machine	1, 9
RTL	Register-Transfer Level	1
UART	Universal Asynchronous Receiver/Transmitter	1